GeoGuess India: Project Documentation

Oct 27, 2026 · @Rishi

Project overview

GeoGuess India is an Android map guessing game: the player sees a photo of an Indian landmark and has 30 seconds to drop a pin where they think it is.

The app measures the distance between the pin and the real location, awards up to 5000 points, reveals the answer with a line on the map, and shows a fun fact. A game has 5 rounds, so the maximum is 25,000 points. Finished games are saved on the phone (SQLite) and to an online leaderboard (Firebase Firestore), and the score can be shared as text.

It was built for the course Advance Android Programming (Symbiosis, T3374).

Item	Value
App name	GeoGuess India
Package	com.example.geoguessindia
Template	Empty Views Activity (not Compose)
minSdk / targetSdk / compileSdk	24 / 34 / 37
Landmarks	50, hosted as places.json on GitHub Pages

Tech stack and changes from the brief

Three choices differ from the original brief: Kotlin instead of Java, osmdroid with Stadia Maps instead of Google Maps, and Firebase for login and the online leaderboard. Confirm them with your lecturer before submitting.

Part	Used in this project	Original brief
Language	Kotlin	Java
Layouts	XML views, Material 3 theme	XML views

Part	Used in this project	Original brief
Map	osmdroid MapView with Stadia Maps tiles	SupportMapFragment, Google Maps SDK
Accounts	Firebase Authentication (email and password)	Not in the brief
Online leaderboard	Cloud Firestore	Not in the brief
Local database	SQLiteOpenHelper (scores, cached_places)	Same
Settings	SharedPreferences	Same
Networking	HttpURLConnection, org.json, AsyncTask	Same
Images	Downloaded manually into Bitmaps	Same
Build files	Gradle Kotlin DSL (build.gradle.kts), AGP 9.4.1	Not specified

Setup we did

The project was set up in five steps, in this order.

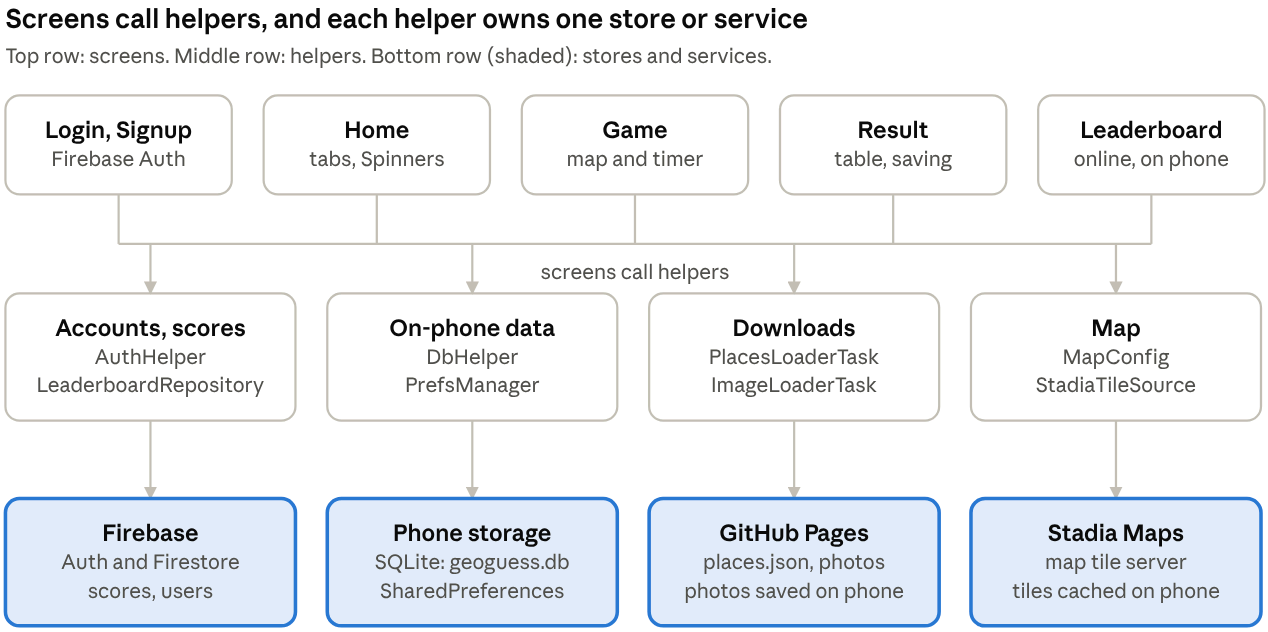
1. **Create the project.** Android Studio, New Project, Phone and Tablet, Empty Views Activity. Name GeoGuess India, package com.example.geoguessindia, Language Kotlin, minimum SDK API 24, Kotlin DSL for Gradle.
2. **Edit Gradle.** The root build.gradle.kts gets the google-services plugin (4.4.4). The app build.gradle.kts adds that plugin, sets targetSdk to 34, adds osmdroid 6.1.20, the Firebase BoM 34.9.0 with Auth and Firestore, and a buildConfigField that reads STADIA_API_KEY from local.properties.
3. **Set up Firebase.** Register the Android app, put google-services.json in the app folder, enable Email/Password sign-in, create Firestore in asia-south1, and publish the security rules (see Accounts and security). A password policy was also set in the console.
4. **Set up Stadia Maps.** Create an account, a property and an API key. Store the key in local.properties as STADIA_API_KEY. The map must show the attribution line for Stadia Maps, OpenMapTiles and OpenStreetMap.

5. **Host the landmark data.** places.json and 50 photos live in a public GitHub repo (geoguess-india) served by GitHub Pages. The app reads reaper22.github.io/geoguess-india/places.json, and every imageUrl points into the same repo.

The photos were downloaded with a script that fetches each landmark's Wikipedia lead image. Keep the credits.csv it writes, because most Wikimedia photos require credit.

Architecture and files

The code is split into screens (activities) in the main package and helpers in five sub-packages: firebase, db, net, map and util, plus model for data classes.



architecture · 5 screens, 4 helpers, 4 stores and services

Screens call helpers, and each helper talks to one store or service. GamePicker, ScoreCalculator and SoundPlayer are plain game logic inside the game and touch no store.

File	What it does	
MainActivity	Home screen: login check, three tabs (Play, Leaderboard, Profile), Spinners, sound switch, quiet cache refresh	
LoginActivity, SignupActivity	Forms, validation, forgot password; call AuthHelper	

File	What it does	
GameActivity	The whole game loop: photo, map, 30-second timer, scoring, reveal, dialogs	
ResultActivity	Score table, saving to SQLite, SharedPreferences and Firestore, share and map intents	
LeaderboardActivity	Online and this-device top 10 with a difficulty filter, clear-scores dialog	
WebViewActivity	The Learn more page	
MapTestActivity, DataTestActivity	Temporary developer test screens, linked from the Profile tab	
firebase/AuthHelper	Sign up, log in, log out, password reset, friendly error messages	
firebase/LeaderboardRepository	Writes and reads the Firestore scores collection	
db/DbHelper	SQLiteOpenHelper with the scores and cached_places tables and all CRUD	
net/PlacesLoaderTask	AsyncTask: downloads and parses places.json	
net/ImageLoaderTask	AsyncTask: downloads each photo into app-private storage and decodes a Bitmap	
map/StadiaTileSource, map/MapConfig	Tile URL pattern for Stadia, and osmdroid setup with its cache in the app folder	
util/ScoreCalculator	Distance and score formula	
util/GamePicker	Chooses the 5 landmarks for a game	
util/PrefsManager	SharedPreferences: name, last difficulty, sound, best score	
util/SoundPlayer	MediaPlayer wrapper for correct and wrong sounds	

File	What it does	
util/AppConfig, util/Extras	The places.json URL, and the intent extra keys	
model/Landmark, RoundResult, ScoreEntry, LocalScore	Plain data classes	

Layouts live in res/layout (one per screen plus item_score.xml), small icons and shapes in res/drawable, colours in values and values-night.

How the app works

One game runs through nine stages. The slow work (downloads) always runs off the main thread so the screen never freezes.

1. **Launch and login.** MainActivity is the launcher. It asks Firebase who is signed in; if nobody, it opens LoginActivity and finishes itself. Firebase keeps the session on the phone, so users stay signed in.
2. **Home.** The three tabs are three panels in one FrameLayout; the TabLayout listener shows one and hides the others. Choosing a difficulty saves it to SharedPreferences. In the background, Home refreshes the landmark cache. Start sends an explicit intent to GameActivity with difficulty and region as extras.
3. **Loading landmarks.** GameActivity reads the SQLite cache first. Only if the cache is empty does PlacesLoaderTask download places.json. That is why the game works offline after one online run.
4. **Choosing rounds.** GamePicker builds a pool by difficulty (Easy: Easy places; Medium: Easy and Medium; Hard: all), narrows it by region, shuffles, and takes 5. Short pools are topped up from other regions.
5. **One round.** ImageLoaderTask shows the photo (from the app folder if already downloaded). Only then does the 30-second CountDownTimer start. Easy mode shows a hint over the photo. The player taps the map; the same pin moves with each tap.
6. **Submit.** Pressing Submit, or the timer reaching zero, calls submitGuess once. A roundActive flag stops a double score if both happen together.
7. **Reveal.** The answer marker and a red line appear, the map zooms to fit both, and a dialog shows distance, points and a fact, with Learn more, Open in Maps and Next round. A sound plays: correct at 3000 points or more, wrong below.

8. **Result.** After round 5, the results go to ResultActivity as intent extras. It builds the TableLayout and saves the game three times: SQLite (history), SharedPreferences (best score) and Firestore (online board). The save runs only on first creation, so a screen recreation cannot save twice.
9. **Leaderboard.** The Online tab asks Firestore for the best 100 scores, filters by difficulty on the phone and keeps 10. The This device tab runs a SQL query with ORDER BY score DESC LIMIT 10.

Offline behaviour

Thing	Offline
Landmark list	Read from the SQLite cache
Photos already seen	Load from the app folder
New photos	Fail; the hint is shown instead
Map tiles	Cached tiles show, the rest stay blank
Scores	Saved to SQLite at once; Firestore sends them when the internet returns
Login	Works only if the user was already signed in

The score system

Each kilometre between the pin and the real place costs 2.5 points from a starting 5000, and the score never goes below 0.

```
\text{score} = \max\left(0,\ 5000 - 2.5 \times d\right), \qquad d = \text{distance in km}
```

The distance comes from Location.distanceBetween(), which measures along the Earth's surface and returns metres; the code divides by 1000. The result is rounded to a whole number for each round, then the rounds are added up.

Worked examples, with the Taj Mahal as the real location (distances are straight-line approximations):

Guess near	Distance	Points
The Taj Mahal itself	0 km	5000
India Gate, Delhi	about 180 km	about 4550
Hawa Mahal, Jaipur	about 220 km	about 4450
Mumbai	about 1060 km	about 2350
Hyderabad	about 1090 km	about 2270
Chennai	about 1580 km	about 1040
Kanyakumari	about 2120 km	0

Useful anchors: 100 km costs 250 points, 800 km gives 3000, and 2000 km or more gives 0 (5000 divided by 2.5).

Rules

- A game has 5 rounds, so the maximum total is 25,000. The Firestore rules cap scores at that number.
- If time runs out with no pin, the round scores 0 and its distance shows as a dash.
- If time runs out with a pin placed, the pin still counts: Submit and the timeout both call the same function.
- The correct sound plays at 3000 points or more (within about 800 km); anything lower plays the wrong sound.
- The formula is linear, as the brief required. Real GeoGuessr-style games use a curve that forgives small errors more.

Accounts and security

Players sign in with email and password through Firebase Authentication. Scores are stored in Cloud Firestore, protected by server-side rules.

Login features

- **Sign up** checks the name, email format, matching passwords and password rules on the phone first, then creates the account, sets the display name and writes a users/{uid} document.

- **Log in** checks the form, then calls Firebase. Errors become plain messages: wrong password, no such user, no internet, weak password.
- **Forgot password** sends a reset email through Firebase. The dialog says "if an account exists", because Firebase hides which emails are registered, so attackers cannot probe for accounts.
- **Password policy** was set in the Firebase console to Require enforcement: minimum 8 characters with upper case, lower case and a number. SignupActivity checks the same rules on the phone for instant feedback. Force upgrade on sign-in was left off.

Firestore data

Collection	Fields
scores	uid, playerName, score, difficulty, rounds, timestamp
users	displayName, bestScore

Security rules in plain words: only signed-in users can read; a score can be created only under your own uid and must be an integer from 0 to 25,000; scores cannot be edited; you can delete only your own; a user document can be written only by its owner.

API keys

- The Stadia key lives in local.properties, which Git ignores, and reaches the code through BuildConfig. It still ends up inside the APK, so keeping it out of Git does not make it secret. If it leaks, regenerate it.
- google-services.json holds a Firebase key that is designed to be public; the Firestore rules protect the data. Keep local.properties and google-services.json out of a public repo.

Known limit. The score is calculated on the phone, so a modified app could send a fake score. The rules limit the range and the owner, but real protection would need a server-side check such as a Cloud Function.

Data layer

Data lives in four places, each chosen for a different job.

Store	Holds	Why this one
SharedPreferences (geoguess_prefs)	player_name, last_difficulty, sound_on, best_score	Small settings as key-value pairs
SQLite (geoguess.db, version 1)	scores, cached_places	Structured data, works offline
Firestore	scores, users	Shared online leaderboard
App-private external storage (files/images)	Downloaded landmark photos	Needs no permission on any Android version

SQLite tables

Table	Columns
scores	id (auto), player, score, difficulty, date
cached_places	id, name, region, lat, lng, image_url, hint, difficulty, fact, learn_more_url

DbHelper operations

Operation	Method
Create	onCreate builds both tables; onUpgrade rebuilds them when the version rises
Insert	insertScore, and cachePlaces (inside one transaction)
Update	updatePlayerName
Delete	deleteScore, clearScores
Query	getTop10 (optional difficulty filter, ORDER BY score DESC LIMIT 10), getCachedPlaces

Networking and JSON. PlacesLoaderTask is an AsyncTask: onPreExecute shows progress on the main thread, doInBackground opens an HttpURLConnection (8-second timeouts), checks for HTTP 200 and parses a JSONArray with org.json, and onPostExecute returns to

the main thread. Each failure has its own message: no internet, timeout, server error, malformed data. Coordinates are validated. A comment in the file shows the modern ExecutorService and Handler version.

Caching. After a successful download, the whole cached_places table is replaced in one transaction. Home refreshes it quietly in the background each time it opens.

Images. ImageLoaderTask looks in getExternalFilesDir("images"). If the file exists it is decoded; if not it is downloaded to that file first. Large photos are shrunk with inSampleSize so they cannot exhaust memory.

UI design

The interface uses a Material 3 DayNight theme with a saffron and green palette, rounded cards, and full dark mode. The restyle changed only XML, so no Kotlin was touched and every view ID stayed the same.

Colour role	Light	Dark
Primary (saffron)	#B24A0A	#FFB68A
Primary container	#FFDBC8	#7B3305
Secondary (green)	#2E7D32	#81C784
Background	#FFF8F4	#18120E
Surface	#FFFFFF	#231B16
Surface variant	#F5E6DC	#3B302A

Screens

- **Login and signup:** a pin logo in a circle, a three-line header, outlined fields with icons and rounded corners, a 56 dp button.
- **Home:** header with logo and tagline, a TabLayout with three tabs, cards for the Spinners and the rules, and a large Start game button.
- **Game:** three pill-shaped stat chips (round, score, timer), a rounded photo card with a gradient hint strip, a rounded map card with the required attribution, and a Submit button.
- **Result:** a tinted hero card with the total (36 sp so it fits on one line), a table card, and Share, Leaderboard and Home buttons.

- **Leaderboard:** tabs for Online and This device, a filter dropdown, and rank-badge rows built from item_score.xml.

Files added: colours in values and values-night, themes.xml, bools.xml (light or dark status bar icons), and drawables for icons, the logo circle, stat pills, rows, badges, the hint gradient and the dropdown background. The template's values-night/themes.xml must be deleted so it does not override the new theme.

Test screens (MapTest and DataTest) keep their original plain layout.

Technical glossary

App structure

Term	Meaning
Activity	One screen of the app, with its own code and layout
Intent	A message that starts another screen or asks another app for help
Explicit intent	Names the exact screen to open, like Home to Game
Implicit intent	Describes a job and lets Android pick the app: share text, open a geo: link
Extras	Small data attached to an intent: difficulty, score, rounds
AndroidManifest.xml	Registers screens, permissions and the theme
Lifecycle	Stages of a screen: onCreate, onResume, onPause, onDestroy
Back stack	History of screens the Back button walks through

Views and events

Term	Meaning
XML layout	A file that describes how a screen looks

Term	Meaning
Spinner, TabLayout, TableLayout, ListView, WebView	Dropdown, row of tabs, grid of rows, scrolling list, mini browser
AlertDialog, Toast	A pop-up that asks or tells; a short fading message
Event listener	Code that runs when something happens, like a tap
CountDownTimer	Built-in countdown that calls back every tick
MediaPlayer	Android's class for playing sounds
dp, sp	Screen-independent sizes for layout and text

Threads and networking

Term	Meaning
Main (UI) thread	The only thread that may touch the screen; slow work on it freezes the app
AsyncTask	Syllabus tool for background work: onPreExecute, doInBackground, onPostExecute
ExecutorService + Handler	The modern replacement for AsyncTask
HttpURLConnection	Android's basic class for web requests
JSON, org.json	A text data format and the library that reads it
Bitmap	A photo held in memory as pixels

Storage

Term	Meaning
SQLite, SQLiteOpenHelper	A database inside the phone, and the class that creates and upgrades it
CRUD	Create, read, update, delete

Term	Meaning
ContentValues, Cursor	A bag of values to write; a pointer for reading rows
SharedPreferences	A tiny key-value store for settings
Cache	A saved copy so nothing is downloaded twice
App-private external storage	A folder only your app can use; needs no permission

Map and Firebase

Term	Meaning
Latitude, longitude	A place's north-south and east-west coordinates
Tile	A small square picture; the map is many tiles
osmdroid, Stadia Maps	The map library, and the tile provider
Overlay, Marker, Polyline	A layer on the map, a pin, a line
Firebase Authentication	Ready-made login system
Firestore, collection, document	Online database; like a table; like a row
Security rules	Server-side rules about who can read or write what

Build and Kotlin

Term	Meaning
Gradle	The build tool that compiles and packages the app
minSdk, targetSdk, compileSdk	Oldest Android supported; version tuned for; version compiled with
API key, local.properties, BuildConfig	A code that identifies your app; a private file for secrets; a class Gradle generates from build settings
object, data class, lateinit	One-instance class; class that mainly holds data; variable set later

Term	Meaning
Lambda	A small block of code passed as an argument, used for callbacks
Null safety (?)	Kotlin forces you to handle a missing value

Syllabus coverage

Five of the seven required feature groups are fully built; external storage is partly built and publishing is still to do. The unit names below follow the original brief; match them to your syllabus wording.

Unit	What implements it	File
Views	ImageView photo, map, Spinners for difficulty and region, TabLayout with 3 tabs, WebView, TableLayout	GameActivity, MainActivity, WebViewActivity, ResultActivity
Views	MediaPlayer sound effects from res/raw	SoundPlayer
Event handling	Map tap to place or move the pin	GameActivity
Event handling	Submit button, CountDownTimer with on-tick updates, Toast, AlertDialog for results and quit	GameActivity
Intents	Explicit: Home to Game to Result to Leaderboard with extras	MainActivity, GameActivity, ResultActivity
Intents	Implicit: share score text, open landmark with a geo: URI	ResultActivity, GameActivity
Internal storage	SharedPreferences: name, last difficulty, sound, best score	PrefsManager
Internal storage	SQLite: scores and cached_places with full CRUD, top 10 and clear	DbHelper

Unit	What implements it	File
Networking and JSON	AsyncTask download and org.json parsing, error handling, progress	PlacesLoaderTask
Networking and JSON	Image download into Bitmaps in the background	ImageLoaderTask
Networking and JSON	Offline use through the SQLite cache	DbHelper, GameActivity
External storage	Images in app-private external storage	ImageLoaderTask
External storage	CSV export, screenshot, runtime permission dialog	Not built yet
Added beyond the brief	Firebase login, signup, forgot password; Firestore leaderboard	AuthHelper, LeaderboardRepository, LoginActivity, SignupActivity, LeaderboardActivity
Publishing	Signed release build, key restriction, icon, store listing	Not done yet

Viva questions and answers

Questions 1 to 10 are the likely ones; 11 to 15 are the harder ones examiners use to test understanding.

1. **What is your project?** A map guessing game. Each round shows a landmark photo and the player has 30 seconds to drop a pin. The app measures the distance, gives up to 5000 points, reveals the answer with a line and a fun fact. A game has 5 rounds, and the score goes to a local SQLite leaderboard and an online Firestore one.
2. **How is the score calculated?** score = max(0, 5000 - distanceKm x 2.5). Distance comes from Location.distanceBetween(), which returns metres; I divide by 1000. 2000 km or more gives 0, and a full game has a maximum of 25,000.
3. **What are explicit and implicit intents, and where did you use them?** An explicit intent names the exact screen: Home to Game to Result to Leaderboard, with difficulty, score and rounds as extras. An implicit intent describes a job and lets Android pick the app:

sharing the score as text (ACTION_SEND) and opening the landmark in a maps app with a geo: URI.

4. **What is AsyncTask and why use it if it is deprecated?** It runs slow work off the main thread, with onPreExecute and onPostExecute on the main thread and doInBackground on a worker. I used it because the syllabus teaches it. It is deprecated because it leaks memory easily; the modern replacement is ExecutorService with Handler, shown in a comment in PlacesLoaderTask.
5. **Why must network work not run on the main thread?** The main thread draws the screen and handles touches, so a download on it freezes the app, and Android throws NetworkOnMainThreadException. Only the main thread can update views, which is why onPostExecute returns to it with the result.
6. **Why three kinds of storage?** Each fits a job. SharedPreferences holds small settings. SQLite holds structured data on the phone: score history and the cached landmark list. Firestore holds the shared online leaderboard.
7. **How does the game work offline?** The landmark list is cached in SQLite and photos are saved in the app's private storage after the first download. Scores go to SQLite immediately, and Firestore keeps the online write on the phone and sends it when the internet returns.
8. **How did you handle network errors?** PlacesLoaderTask catches each failure: UnknownHostException and ConnectException mean no internet, SocketTimeoutException means timeout, a non-200 code means a server error, and JSONException means malformed data. If a cache exists, the game still starts.
9. **How does CountDownTimer work in your game?** It is created with 30,000 ms and a 1,000 ms interval. onTick updates the text, which turns red at 10 seconds; onFinish calls submitGuess(timedOut = true). It starts only after the photo loads, so a slow download does not cost the player time.
10. **How does a map tap become a guess?** osmdroid's MapEventsOverlay turns the touched pixel into a GeoPoint (latitude and longitude). The first tap creates a marker and later taps move it. This is the equivalent of OnMapClickListener. Submit stays disabled until the first tap.
11. **Is your API key secure?** Partly. The Stadia key is in local.properties, which Git ignores, and Gradle injects it through BuildConfig, so it stays out of the repository. It still ends up inside the APK, so anyone could extract it. The real protection is regenerating it if leaked. The Firebase config is designed to be public and is protected by Firestore rules.

12. **Can a player cheat on the leaderboard?** In principle yes: the score is calculated on the phone, so a modified app could send a fake number. The Firestore rules limit the damage (signed in, own uid only, integer 0 to 25,000). Real protection needs a server-side check, such as a Cloud Function.
13. **Why check savedInstanceState == null in ResultActivity?** Android can recreate an activity, for example when dark mode or the language changes. onCreate would run again and save the same game twice. The check makes the three saves happen only the first time.
14. **What does roundActive do?** It is a guard. The player could press Submit at the instant the timer ends, scoring the round twice. submitGuess starts with if (!roundActive) return and sets it to false at once. Map taps between rounds are also ignored while it is false.
15. **How does the project differ from the brief?** Three agreed changes: Kotlin instead of Java, osmdroid with Stadia Maps instead of SupportMapFragment and Google Maps, and Firebase added for login and the online leaderboard (SQLite is kept). The CSV export, screenshot and runtime permission flow are not built; photos use app-private external storage, which needs no permission. On scoped storage: Android 10 and above writes public files through MediaStore with no permission, while Android 9 and below needs WRITE_EXTERNAL_STORAGE.

Team of two: dividing the work

Split by feature area, so each person owns whole files and the two only meet at a few agreed points.

	Person A: Gameplay and map	Person B: Accounts, data and leaderboard
Screens	GameActivity, ResultActivity, WebViewActivity	MainActivity, LoginActivity, SignupActivity, LeaderboardActivity
Helpers	GamePicker, ScoreCalculator, SoundPlayer, MapConfig, StadiaTileSource, RoundResult	AuthHelper, LeaderboardRepository, DbHelper, PrefsManager, PlacesLoaderTask, ImageLoaderTask, AppConfig
Models	RoundResult	Landmark, LocalScore, ScoreEntry

	Person A: Gameplay and map	Person B: Accounts, data and leaderboard
Layouts	game, result, webview	main, login, signup, leaderboard, item_score
Outside the code	Stadia account and key, sound files, MapTestActivity	Firebase console and rules, places.json, 50 photos, GitHub Pages, DataTestActivity
Syllabus units	ImageView, map events, CountDownTimer, AlertDialog, Toast, MediaPlayer, WebView, TableLayout, implicit intents	TabLayout, Spinner, SQLite CRUD, SharedPreferences, AsyncTask, JSON, external storage, Firebase

Done together: colours, theme and strings, the manifest, the Gradle files, Extras.kt, the test plan, the report and viva practice. If B's side feels heavier, move MainActivity to A.

Agree on day one

- The Landmark data class and the Extras keys; do not change them without telling each other.
- B provides getCachedPlaces, cachePlaces, insertScore, updateBestScore, PlacesLoaderTask, ImageLoaderTask, currentName and submitScore.
- A provides the game, which Home starts with difficulty and region extras, and which ends by starting ResultActivity with the score extras.
- Until B's loader is ready, A tests with 5 hard-coded Landmark objects. B tests with DataTestActivity.

Order of work

1. Together: create the project, Gradle, Firebase and Stadia setup, agree the data class and extras, push a first commit.
2. In parallel: A builds map, game loop, scoring and dialogs; B builds login, SQLite, networking, hosting and Firestore.
3. Together: connect Home to Game to Result to Leaderboard and fix what breaks.
4. Together: UI polish, the 10 test cases, release build, report and viva practice.

Git workflow. One repo with main plus feature/game and feature/accounts-data branches; merge by pull request after the other person reads it. AndroidManifest.xml, strings.xml, activity_main.xml and build.gradle.kts are edited by both, so add lines in your

own commented block and merge often. Never commit local.properties or google-services.json; share them privately and add the teammate under Firebase Users and permissions.

Both must know everything. A viva asks each person about the whole app. After step 3, spend an hour explaining your half to each other, and write five viva questions about the other's half. Include a contribution table in the report.

Problems we hit and fixes

Problem	Cause	Fix
Build failed: Firebase Auth compiled with Kotlin 2.4, project expects 2.2	Firebase BoM 34.17.0 pulled Auth 25.0.0, newer than the project's Kotlin compiler	Use BoM 34.9.0; fallback is the skip-metadata-version-check compiler flag
Template generated targetSdk 37, brief says 34	New Android Studio defaults	Set targetSdk = 34 by hand; raise it again before publishing
Package is com.example.geoguessindia	Template default	Fine for the course; Play Store rejects com.example, so change applicationId before publishing
places.json returned 404	GitHub Pages was not switched on; the account-level Pages page is for custom domains only	Open the repo's own Settings, then Pages, deploy from branch main, root
Game score wrapped onto two lines on the Result screen	tvTotal text size was 52 sp	Set it to 36 sp
Weak passwords still accepted	Password policy was on Notify enforcement	Switch to Require enforcement and mirror the rules in SignupActivity

Problem	Cause	Fix
Wrong password message said 6 characters	friendlyError ignored the policy text	Show the weak-password exception's reason instead
Dark mode kept purple colours	The template's values-night/themes.xml overrode the new theme	Delete that file; night colours come from values-night/colors.xml
Wrong sanity check in the map test	An early note said Chennai scores about 600	Chennai is about 1580 km from the Taj Mahal, so about 1040 points
Photos for 50 landmarks	Searching by hand is slow	A script fetches each landmark's Wikipedia lead image and writes credits.csv; check each image for collages or signs that give the answer away

Limitations and what is left

The app plays end to end; what remains is one brief feature, the release steps and the paperwork.

Still to do

- [ ] CSV export of the leaderboard to public storage (brief feature 6; ask your lecturer if it is required)
- [ ] Result screenshot save and the runtime permission dialog with its rationale and denial handling
- [ ] Test plan with 10 test cases
- [ ] Signed release build (AAB or APK), version and icon, store listing
- [ ] Raise targetSdk and compileSdk for publishing and retest; change the applicationId from com.example
- [ ] Restrict API keys and check Stadia's terms before publishing
- [ ] Remove or keep MapTestActivity and DataTestActivity before submitting
- [ ] Check each landmark's hint and fact against a source; they were written from memory

- [ ] Review every downloaded photo and keep the credits for Wikimedia images

Known limitations

- Scores are calculated on the phone, so a modified app could send fake scores.
- Progress inside a game is not saved; quitting mid-game loses it.
- AsyncTask is deprecated. It works and matches the syllabus; the replacement is shown in a comment.
- The Stadia key is inside the APK and cannot be restricted by package name the way a Google key can.
- Offline, only cached map tiles and already downloaded photos show.
- Regions Central, Northeast and Islands have few landmarks (2, 3 and 1), so the region filter often tops up from other regions.